

Product Documentation

Intelligent ML-Driven Decision Override & Risk Control System

In this project, we build an Intelligent Decision System where the machine learning model does not take the final decision, but only provides a suggestion with a confidence score. Whenever a high risk decision occurs (such as a high-value transaction, refund, or approval), the ML model first generates a prediction along with its confidence. Then, a decision control layer evaluates whether the model confidence is strong enough, whether the risk level is acceptable, and whether any business rules are violated. If all conditions are safe, the system automatically approves the decision; however, if the confidence is low, the risk is high, or a rule is violated, the system overrides the ML output and either sends the case for manual review or directly rejects it. For every decision, the system also provides a clear explanation of why the decision was made, ensuring that the business does not blindly trust ML but instead operates with controlled, explainable, and safe automation.

1. Company Overview

Company Name

DecisioCore Systems Pvt. Ltd.

About the Company

DecisioCore builds **decision intelligence platforms** for enterprises where **wrong decisions are expensive**.

Our clients operate in:

- FinTech
- Insurance
- E-commerce
- Logistics

These companies already use ML models —
but **they fail because decisions are blindly trusted**.

2. The Real, Critical Business Problem

What Actually Goes Wrong in Real Companies

Most ML systems fail in production due to **3 serious reasons**:

1. **Model confidence is ignored**
2. **Edge cases are not handled**
3. **No controlled fallback when ML is uncertain**

As a result:

- Wrong approvals happen
- Business losses increase
- Teams lose trust in ML

3. Core Problem Statement (Very Senior)

Build a **Decision Control System** that does NOT blindly trust ML predictions but intelligently

decides: • When to trust ML

- When to override ML
- When to escalate to human review

This system must behave like a **senior decision-maker**, not a script.

4. What Makes This Project Different (Important)

This is **NOT**:

- Just training a model
- Just exposing an API
- Just predicting something

This **IS**:

- Decision confidence analysis
- Risk-aware logic
- Failure-aware system design
- Explainable overrides

5. High-Level Use Case (Generic but Real)

The system is used for **high-risk automated decisions**, such as:

- Order fraud checks
- Refund approvals
- High-value customer actions
- Risk-based approvals

(Exact domain can change — system stays same.)

6. Input Data (From MySQL)

System receives structured event

data: - user_id

- transaction_amount

- frequency_last_24h

- historical_success_rate

- anomaly_score

- model_prediction

- model_confidence

Notice: **model confidence is a FIRST-CLASS input.**

7. Step-by-Step Logical Solution (Critical

Thinking)

Step 1: ML Prediction (Not Final)

ML model predicts outcome:

```
{  
  "prediction": "ALLOW",  
  "probability": 0.58  
}
```

Important:

- Prediction is **not accepted directly** •

Probability is **evaluated critically**

Step 2: Confidence Zone Classification

Model confidence is mapped into zones:

Probability Confidence Zone

≥ 0.80 HIGH_CONFIDENCE

0.55 – 0.80 MEDIUM_CONFIDENCE

< 0.55 LOW_CONFIDENCE

Step 3: Risk Context Evaluation

Even with high confidence, risk context

matters: - High transaction amount?

- Abnormal behavior frequency?

- User historical reliability?

Example:

IF amount > ₹50,000 AND confidence <

0.85 → escalate

Step 4: Decision Override Logic

This is the **core of the system**.

Rules:

IF confidence HIGH AND risk LOW

→ AUTO APPROVE

IF confidence MEDIUM AND risk

MEDIUM → MANUAL REVIEW

IF confidence LOW

→ AUTO REJECT OR ESCALATE

IF ML approves BUT rule violates

→ OVERRIDE ML

System is allowed to disagree with ML.

Step 5: Human-in-the-Loop Trigger

For uncertain cases:

- Decision is **paused**
- Case is tagged for review

- ML outcome is stored but not applied

This is **enterprise-grade design**.

Step 6: Explanation Generator

Every decision returns **WHY**:

```
{  
  "final_decision": "MANUAL_REVIEW",  
  "ml_prediction": "ALLOW",  
  "confidence": 0.61,  
  "override_reason": [  
    "Medium confidence score",  
    "High transaction value",  
    "Unusual activity spike"  
  ]  
}
```

This builds **trust**.

8. REST API Design

Decision API

POST /decision/evaluate

This API:

- Accepts ML output + raw data •
- Applies control logic
- Returns final decision

ML is **not the owner** — system is.

9. System Guarantees

This system guarantees:

- No blind ML decisions
- Controlled risk exposure

- Auditable decisions
- Explainable overrides

10. Final Output

(Realistic) {

"decision_id": "DEC-2026-00192",

"final_decision": "REJECT",

"ml_prediction": "APPROVE",

"confidence": 0.53,

"decision_path": [

"Low confidence detected",

"High transaction amount",

"Risk threshold exceeded"]

}